LeetCode Practice

12 Medium: Greedy

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

12
MEDIUM

Patterns (in order)

1. Greedy (12)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

12 questions | tap a title to jump to it

Medium questions (12)	Go >
Greedy (12)	**Go >**
M 1. Gas Station	LC 134
M 2. Largest Number	LC 179
M 3. Wiggle Sort	LC 280
M 4. Increasing Triplet Subsequence	LC 334
M 5. Minimum Number of Arrows to Burst Balloons	LC 452
M 6. Dota2 Senate	LC 649
M 7. Maximum Swap	LC 670
M 8. Hand of Straights	LC 846
M 9. Array of Doubled Pairs	LC 954
M 10. Minimum Deletions to Make Character Frequencies Unique	LC 1647
M 11. Merge Triplets to Form Target Triplet	LC 1899
M 12. Longest Palindrome by Concatenating Two Letter Words	LC 2131

Medium questions (12)

MEDIUM | TOPIC 1 OF 1

Greedy

12 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Gas Station LC 134
2. Largest Number LC 179
3. Wiggle Sort LC 280
4. Increasing Triplet Subsequence LC 334
5. Minimum Number of Arrows to Burst Balloons LC 452
6. Dota2 Senate LC 649
7. Maximum Swap LC 670
8. Hand of Straights LC 846
9. Array of Doubled Pairs LC 954
10. Minimum Deletions to Make Character Frequencies Unique LC 1647
11. Merge Triplets to Form Target Triplet LC 1899
12. Longest Palindrome by Concatenating Two Letter Words LC 2131

MEDIUM | Greedy | LeetCode #134 | 1 of 12 | 1/12

Gas Station

QUESTION

There are n gas stations along a circular route, where the amount of gas at the ith station is gas[i].

You have a car with an unlimited gas tank and it costs cost[i] of gas to travel from the ith station to its next (i + 1)th station. You begin the journey with an empty tank at one of the gas stations.

Given two integer arrays gas and cost, return the starting gas station's index if you can travel around the circuit once in the clockwise direction, otherwise return -1. If there exists a solution, it is guaranteed to be unique.

Example 1

Input	`gas = [1,2,3,4,5], cost = [3,4,5,1,2]`
Output	`3`
Explanation	*Start at station 3 (index 3) and fill up with 4 unit of gas. Your tank = 0 + 4 = 4* *Travel to station 4. Your tank = 4 - 1 + 5 = 8* *Travel to station 0. Your tank = 8 - 2 + 1 = 7* *Travel to station 1. Your tank = 7 - 3 + 2 = 6* *Travel to station 2. Your tank = 6 - 4 + 3 = 5* *Travel to station 3. The cost is 5. Your gas is just enough to travel back to station 3.* *Therefore, return 3 as the starting index.*

Example 2

Input	`gas = [2,3,4], cost = [3,4,3]`
Output	`-1`
Explanation	*You can't start at station 0 or 1, as there is not enough gas to travel to the next station.* *Let's start at station 2 and fill up with 4 unit of gas. Your tank = 0 + 4 = 4* *Travel to station 0. Your tank = 4 - 3 + 2 = 3* *Travel to station 1. Your tank = 3 - 3 + 3 = 3* *You cannot travel back to station 2, as it requires 4 unit of gas but you only have 3.* *Therefore, you can't travel around the circuit once no matter where you start.*

CONSTRAINTS

- n == gas.length == cost.length
- $1 <= n <= 10^5$
- $0 <= gas[i], cost[i] <= 10^4$
- The input is generated such that the answer is unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

n gas stations in a circle. gas[i] = fuel at i, cost[i] = fuel to reach i+1.

Start with empty tank. Find the starting station to complete the circuit.

If impossible return -1; the solution is guaranteed unique if it exists. Right?"

"A few quick questions:

Can we start at any station? Yes. Must return to the starting station? Yes."

"Let me walk: gas=[1,2,3,4,5], cost=[3,4,5,1,2] -> start at station 3.

0+4-1=3 -> +5=8 -> +1=9 -> +2=11 -> +3=14 -> -4=10 -> return to 3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for s in range(n):
    simulate the full circle and check the tank
        never goes negative
return the first start that works
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Simulates n steps from each of n starts. If total gas is at least total cost a start exists; reset the start after any prefix that goes negative: one pass, O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is trying every start independently.

Two greedy observations:

1. If total gas < total cost, no solution — return -1.

2. If we run out of gas starting at s after reaching index i,

no station between s and i can be a valid start (they'd all begin with less gas).

So reset candidate to i+1 and continue.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _134_GasStation:
    def canCompleteCircuit(self, gas, cost):
        total_surplus = 0
```

```
        current_tank = 0
        candidate = 0
        for i in range(len(gas)):
            net = gas[i] - cost[i]
            total_surplus += net
            current_tank += net
            if current_tank < 0:
                candidate = i + 1
                current_tank = 0
        return candidate if total_surplus >= 0 else -1
```

PHASE 5 TEST & DEBUG

"Let me trace canCompleteCircuit([1,2,3,4,5],[3,4,5,1,2])."

If total gas >= total cost -> solution exists. Find starting point.

net=[-2,-2,-2,3,3]. total=0>=0 -> solution exists.

Walk: tank=0. i=0: tank+=net[0]=-2<0 -> reset. start=1, tank=0.

i=1: tank+=net[1]=-2<0 -> reset. start=2, tank=0.

i=2: tank+=net[2]=-2<0 -> reset. start=3, tank=0.

i=3: tank+=net[3]=3. i=4: tank+=net[4]=3. tank=6>0 -> start=3 ok

Return 3 ok (start at station 3: 4gas-1cost=3; +5-2=6; +1-3=4; +2-4=2; +3-5=0. Complete ok)

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
single pass	three integer variables

The key insight: if total gas >= total cost, a valid start always exists;

the greedy candidate is guaranteed to be it.

Follow-up: what if there are multiple valid starting points? The problem guarantees uniqueness

but the greedy still finds the first valid one.

Follow-up: minimum cost to fill gas (variant) — different problem, uses a monotonic stack.

MEDIUM | Greedy | LeetCode #179 | 2 of 12 | 2/12

Largest Number

QUESTION

Given a list of non-negative integers nums, arrange them such that they form the largest number and return it.

Since the result may be very large, so you need to return a string instead of an integer.

Example 1	
Input	`nums = [10,2]`
Output	`"210"`

Example 2	
Input	`nums = [3,30,34,5,9]`
Output	`"9534330"`

CONSTRAINTS

- 1 <= nums.length <= 100
- 0 <= nums[i] <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Arrange non-negative integers so they form the largest number. Return as a string.

Edge: if the result starts with '0', return '0' (e.g. [0,0] -> '0'). Right?"

"A few quick questions:

Values can be 0? Yes. Return a string because the result may be very large? Yes."

"Let me walk: nums=[10,2] -> '210' > '102' -> '210' ok

nums=[3,30,34,5,9] -> '9534330' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for ordering in orderings(numbers_as_strings):
    join them
return the largest joined string
```

TIME	SPACE
O(n! * n)	O(n)

WHY IT IS SLOW

n! orderings. Sort the strings with the comparator 'a + b is bigger than b + a' and join them: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is generating all permutations.

Custom comparator: to decide whether a comes before b, compare str(a)+str(b) vs str(b)+str(a).

Sort all numbers by this comparator; concatenate; strip leading zeros if needed.

Time: O(n log n). Space: O(n) for the string array."

Key insight: the custom comparator (compare str(a)+str(b) vs str(b)+str(a)) is transitive and produces the unique lexicographically largest concatenation — standard sort then applies.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from functools import cmp_to_key
class _179_LargestNumber:
    def largestNumber(self, nums):
        strs = [str(n) for n in nums]
        def compare(a, b):
            return (1 if b + a > a + b else -1)
        strs.sort(key=cmp_to_key(compare))
        result = ''.join(strs)
        return '0' if result[0] == '0' else result
```

PHASE 5 TEST & DEBUG

"Let me trace the greedy choice and show why that choice can't hurt the optimal result."

nums=[10,2]: compare('10','2'): '210' vs '102' -> '210'>'102' -> 2 before 10.

sorted=['2','10']. -> '210' ok

nums=[3,30,34,5,9]: compare determines order. '9'>'5'>'34'>'3'>'30'.

-> '9534330' ok

nums=[0,0]: sorted=['0','0']. result='00'. result[0]='0' -> return '0' ok

The leading-zero check handles [0,0] without special-casing

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n * k)	O(n*k)
sorting with O(k) string comparisons	string array

The comparator is transitive and total-ordering — provably correct for any input.

Follow-up: smallest number from array — same comparator, ascending sort.

Follow-up: if numbers can be very large (BigInteger) — convert to strings first anyway.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Greedy | LeetCode #280 | 3 of 12 | 3/12

Wiggle Sort

QUESTION

Given an integer array nums, reorder it such that nums[0] <= nums[1] >= nums[2] <= nums[3]....

You may assume the input array always has a valid answer.

Example 1	
Input	`nums = [3,5,2,1,6,4]`
Output	`[3,5,1,6,2,4]`
Explanation	*[1,6,2,5,3,4] is also accepted.*

Example 2	
Input	`nums = [6,6,5,6,3,8]`
Output	`[6,6,5,6,3,8]`

CONSTRAINTS

- 1 <= nums.length <= $5 * 10^4$
- 0 <= nums[i] <= 10^4
- It is guaranteed that there will be an answer for the given input nums.
- Follow up: Could you solve the problem in O(n) time complexity?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Reorder nums in-place so: nums[0] <= nums[1] >= nums[2] <= nums[3] >= ...

Multiple valid answers exist. Right?"

"A few quick questions:

Any valid wiggle order is accepted? Yes. Modify in-place, no return? Yes."

"Let me walk: nums=[3,5,2,1,6,4] -> [3,5,1,6,2,4] ok (one valid answer)

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
sort nums
then swap neighbouring pairs (positions 1 and 2, 3
    and 4, ...) so the wiggle holds
```

TIME	SPACE
O(n log n)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

The sort is unnecessary. One pass that swaps nums[i] with nums[i - 1] whenever the required less-than or greater-than order is broken is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n log n) sort.

Greedy one-pass O(n): check each index i.

Odd index (i%2==1): nums[i] should be >= nums[i-1]. If not, swap.

Even index (i%2==0) for i>0: nums[i] should be <= nums[i-1]. If not, swap.

A local swap always fixes the current violation without breaking prior pairs.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _280_WiggleSort:
    def wiggleSort(self, nums):
        for i in range(1, len(nums)):
            if (i % 2 == 1 and nums[i] < nums[i-1]) or
            (i % 2 == 0 and nums[i] > nums[i-1]):
                nums[i], nums[i-1] = nums[i-1], nums[i]
```

PHASE 5 TEST & DEBUG

"Let me trace the greedy choice and show why that choice can't hurt the optimal result."

nums=[3,5,2,1,6,4]:

i=1(odd): 5>=3 ok. i=2(even): 2<=5 ok. i=3(odd): 1<2 -> swap -> [3,5,1,2,6,4].

i=4(even): 6>2 -> swap -> [3,5,1,6,2,4]. i=5(odd): 4>=2 ok. -> [3,5,1,6,2,4] ok

nums=[6,6,5,6,3,8]: all conditions already satisfied -> no swaps ok

The swap at position i only affects the pair (i-1, i), never undoes earlier swaps

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)
single pass	in-place swaps

Greedy correctness: swapping adjacent elements to fix one condition never invalidates the

previously satisfied condition — because the new value at i-1 satisfies the prior pair.

Follow-up: Wiggle Sort II (#324) — strict inequalities nums[0]<nums[1]>nums[2]<...

PHASE 6 COMPLEXITY & FOLLOW-UP

Requires median partitioning to guarantee strict separation.

Follow-up: count wiggle subsequences (#376) — DP/greedy, no reordering needed.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Greedy | LeetCode #334 | 4 of 12 | 4/12

Increasing Triplet Subsequence

QUESTION

Given an integer array nums, return true if there exists a triple of indices (i, j, k) such that i < j < k and nums[i] < nums[j] < nums[k]. If no such indices exists, return false.

Example 1	
Input	`nums = [1,2,3,4,5]`
Output	`true`
Explanation	*Any triplet where i < j < k is valid.*

Example 2	
Input	`nums = [5,4,3,2,1]`
Output	`false`
Explanation	*No triplet exists.*

Example 3	
Input	`nums = [2,1,5,0,4,6]`
Output	`true`
Explanation	*One of the valid triplet is (1, 4, 5), because nums[1] == 1 < nums[4] == 4 < nums[5] == 6.*

CONSTRAINTS

- 1 <= nums.length <= $5 * 10^5$
- 2^{31} <= nums[i] <= $2^{31} - 1$
- Follow up: Could you implement a solution that runs in O(n) time complexity and O(1) space complexity?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return true if there exist indices i<j<k such that nums[i]<nums[j]<nums[k].

Must be a STRICTLY increasing triplet subsequence. Right?"

"A few quick questions:

Not necessarily consecutive? Correct — indices just need to be in order.

Follow-up asks for O(n) time, O(1) space?"

"Let me walk: nums=[2,1,5,0,4,6] -> 1<4<6 ok. nums=[5,4,3,2,1] -> false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        for k in range(j_+_1,_n):
            if nums[i] < nums[j] < nums[k]:
                return True
return False
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

n^3/6 triples. Track the smallest value and the smallest 'second' value so far; any number above both completes a triplet: O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the cubic search.

Greedy with two minimums: track the smallest (first) and second smallest (second) seen so far.

If any element > second, we have a valid triplet.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _334_IncreasingTripletSubsequence:
    def increasingTriplet(self, nums):
        first = second = float('inf')
        for num in nums:
            if num <= first:
                first = num
            elif num <= second:
                second = num
            else:
                return True
        return False
```

PHASE 5 TEST & DEBUG

"Let me trace the greedy choice and show why that choice can't hurt the optimal result."

nums=[2,1,5,0,4,6]:

2->first=2. 1->first=1. 5>first,5>second=inf->second=5. 0->first=0. 4>first=0,4<second=5->second=4.

6>first=0,6>second=4 -> True ok

nums=[5,4,3,2,1]: each updates first only -> second stays inf -> False ok

PHASE 5 TEST & DEBUG

nums=[1,2,3]: 1->first=1. 2>first->second=2. 3>second -> True ok

First and second represent the two smallest elements for the first two positions

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

Note: first and second may not be adjacent in the actual triplet — they're virtual minimums.

Follow-up: Longest Increasing Subsequence (#300) — LIS of any length; patience sort O(n log n).

Follow-up: find the actual triplet indices — use LIS with backtracking.

MEDIUM | Greedy | LeetCode #452 | 5 of 12 | 5/12

Minimum Number of Arrows to Burst Balloons

QUESTION

There are some spherical balloons taped onto a flat wall that represents the XY-plane. The balloons are represented as a 2D integer array points where points[i] = [x_{start}, x_{end}] denotes a balloon whose horizontal diameter stretches between x_{start} and x_{end}. You do not know the exact y-coordinates of the balloons.

Arrows can be shot up directly vertically (in the positive y-direction) from different points along the x-axis. A balloon with x_{start} and x_{end} is burst by an arrow shot at x if x_{start} <= x <= x_{end}. There is no limit to the number of arrows that can be shot. A shot arrow keeps traveling up infinitely, bursting any balloons in its path.

Given the array points, return the minimum number of arrows that must be shot to burst all balloons.

Example 1

Input	`points = [[10,16],[2,8],[1,6],[7,12]]`
Output	`2`
Explanation	*The balloons can be burst by 2 arrows: - Shoot an arrow at x = 6, bursting the balloons [2,8] and [1,6]. - Shoot an arrow at x = 11, bursting the balloons [10,16] and [7,12].*

Example 2

Input	`points = [[1,2],[3,4],[5,6],[7,8]]`
Output	`4`
Explanation	*One arrow needs to be shot for each balloon for a total of 4 arrows.*

Example 3

Input	`points = [[1,2],[2,3],[3,4],[4,5]]`
Output	`2`
Explanation	*The balloons can be burst by 2 arrows: - Shoot an arrow at x = 2, bursting the balloons [1,2] and [2,3]. - Shoot an arrow at x = 4, bursting the balloons [3,4] and [4,5].*

CONSTRAINTS

- 1 <= points.length <= 10^5
- points[i].length == 2
- 2^{31} <= x_{start} < x_{end} <= 2^{31} - 1

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Each balloon spans [x_start, x_end]. An arrow shot at x bursts all balloons spanning that x.

Find the minimum number of arrows to burst all balloons. Right?"

"A few quick questions:

Arrows travel vertically, touching the boundary counts? Yes — inclusive."

"Let me walk: points=[[10,16],[2,8],[1,6],[7,12]] -> 2 arrows ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets(arrow_positions):
    if every balloon is hit:
        best = min(best, subset)
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

Tries every set of shots. Sort balloons by right end and shoot at the first one's end; skip every balloon that starts at or before it: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sort by end, greedy arrow placement — O(n log n). Walk-through:"

Sort balloons by their END coordinate.

`arrows = 1. shoot_at = balloons[0][1] (first arrow at end of first balloon).`

For each subsequent balloon (start, end):

If start > shoot_at: <- this balloon starts AFTER the current arrow -> can't burst it.

`arrows += 1; shoot_at = end (shoot new arrow at THIS balloon's end).`

<- If start <= shoot_at: current arrow already bursts this balloon too. No new arrow.

WHY sort by end: shooting at the END of the earliest-ending balloon covers the maximum

number of overlapping balloons that also end near that point.

Time: O(n log n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _452_MinArrowsToBurstBalloons:
    def findMinArrowShots(self, points):
        points.sort(key=lambda x: x[1])
        arrows = 0
        arrow_pos = float('-inf')
        for start, end in points:
            if start > arrow_pos:
                arrows += 1
                arrow_pos = end
        return arrows
```

PHASE 5 TEST & DEBUG

"Let me trace findMinArrowShots([[10,16],[2,8],[1,6],[7,12]])."

Sort by end: [[1,6],[2,8],[7,12],[10,16]]

arrow=6 (first balloon's end). Count=1

[2,8]: 2<=6 -> arrow=6 covers it (min(end,6)=6>=start=2)

[7,12]: 7>6 -> new arrow at 12. Count=2

[10,16]: 10<=12 -> covered. Count stays 2. Return 2 ok

WHY sort by end: greedily shoot at rightmost point of first balloon to overlap maximum.

Edge: single balloon -> 1 arrow. No gaps between sorted balloons -> 1 arrow covers all.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	O(1)

Follow-up: Non-overlapping Intervals (#435) — same greedy, count removed intervals.

Follow-up: if arrows have a width, each arrow covers [x-w, x+w]; same greedy with adjusted overlap check.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Greedy | LeetCode #649 | 6 of 12 | 6/12

Dota2 Senate

QUESTION

In the world of Dota2, there are two parties: the Radiant and the Dire.

The Dota2 senate consists of senators coming from two parties. Now the Senate wants to decide on a change in the Dota2 game. The voting for this change is a round-based procedure. In each round, each senator can exercise one of the two rights:

CONSTRAINTS

- Ban one senator's right: A senator can make another senator lose all his rights in this and all the following rounds.
- Announce the victory: If this senator found the senators who still have rights to vote are all from the same party, he can announce the victory and decide on the change in the game.

Given a string senate representing each senator's party belonging. The character 'R' and 'D' represent the Radiant party and the Dire party. Then if there are n senators, the size of the given string will be n.

The round-based procedure starts from the first senator to the last senator in the given order. This procedure will last until the end of voting. All the senators who have lost their rights will be skipped during the procedure.

Suppose every senator is smart enough and will play the best strategy for his own party. Predict which party will finally announce the victory and change the Dota2 game. The output should be "Radiant" or "Dire".

Example 1

Input	`senate = "RD"`
Output	`"Radiant"`
Explanation	*The first senator comes from Radiant and he can just ban the next senator's right in round 1.* *And the second senator can't exercise any rights anymore since his right has been banned.* *And in round 2, the first senator can just announce the victory since he is the only guy in the senate who can vote.*

QUESTION

Example 2

Input	`senate = "RDD"`
Output	`"Dire"`
Explanation	*The first senator comes from Radiant and he can just ban the next senator's right in round 1.* *And the second senator can't exercise any rights anymore since his right has been banned.* *And the third senator comes from Dire and he can ban the first senator's right in round 1.* *And in round 2, the third senator can just announce the victory since he is the only guy in the senate who can vote.*

- n == senate.length
- 1 <= n <= 10^4
- senate[i] is either 'R' or 'D'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Senators from Radiant(R) and Dire(D) take turns banning the next opponent senator or declaring victory.

Simulate rounds until one party has no senators left; return the winning party. Right?"

"A few quick questions:

A banned senator is permanently removed from all future rounds? Yes. Process continues round-robin in original order."

"Let me walk: senate='RD' -> R bans D's only senator -> Radiant wins ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
go around the circle repeatedly: each active senator
    bans the next active senator of the other party
    (found by scanning)
if one party is left:
    break
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Scans to find the next opponent every time. Two queues of senator indices: compare the fronts, the smaller index wins, bans the other and re-enters with index + n. O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Greedy queue simulation — O(n log n). Walk-through:"

`radiant_queue = sorted radiant senator indices.`
`dire_queue = sorted dire senator indices.`

While both queues non-empty:

`r = radiant_queue.popleft(). d = dire_queue.popleft().`

If r < d: Radiant senator bans Dire -> radiant_queue.append(r + n). (comes back next round)

Else: Dire senator bans Radiant -> dire_queue.append(d + n).

Return 'Radiant' if radiant_queue else 'Dire'.

WHY add n: the senator re-enters the queue n positions later (circular, n = total senators).

WHY compare indices: earlier index = acts first in this round -> bans the other.

Whichever senator comes first bans the other's earliest senator.

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import deque
class _649_Dota2Senate:
    def predictPartyVictory(self, senate):
        n = len(senate)
        radiant = deque(i for i, s in enumerate(senate) if
 s == 'R')
        dire = deque(i for i, s in enumerate(senate) if s
== 'D')
        while radiant and dire:
            r_idx = radiant.popleft()
            d_idx = dire.popleft()
            if r_idx < d_idx:
                radiant.append(r_idx + n)
            else:
                dire.append(d_idx + n)
        return 'Radiant' if radiant else 'Dire'
```

PHASE 5 TEST & DEBUG

"Let me trace the greedy choice and show why that choice can't hurt the optimal result."

senate='RD': radiant=[0], dire=[1]. r_idx=0,d_idx=1. 0<1->radiant bans dire's senator, radiant=[0+2=2].

dire empty -> loop ends -> 'Radiant' ok

Re-appending index+n preserves relative turn order for the next round without re-scanning

Edge: 'RRR' -> each Radiant senator bans a non-existent Dire -> Dire queue empties immediately -> 'Radiant' ok.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(n)

Follow-up: Task Scheduler (#621) — different greedy/queue simulation pattern.

Follow-up: simulate with k parties instead of 2 — generalize to k queues, ban round-robin.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Greedy | LeetCode #670 | 7 of 12 | 7/12

Maximum Swap

QUESTION

You are given an integer num. You can swap two digits at most once to get the maximum valued number.

Return the maximum valued number you can get.

Example 1	
Input	num = 2736
Output	7236
Explanation	*Swap the number 2 and the number 7.*

Example 2	
Input	num = 9973
Output	9973
Explanation	*No swap.*

CONSTRAINTS

- 0 <= num <= 10^8

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Swap two digits of a non-negative integer to make the largest possible number.

At most one swap. Right?"

"A few quick questions:

Return the result as integer? Yes. No leading zeros after swap? Not guaranteed but swap valid digits."

"Let me walk: num=2736 -> swap 7 and 6? No, swap 2 and 7 -> 7236. or swap 2 and 6 -> 6732. Best: 7236 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (a, b) in pairs(digit):
    swap them
    best = max(best, candidate)
```

TIME	SPACE
O(d^3)	O(d)

WHY IT IS SLOW

d^2 swaps, each producing a comparison of d digits. Record the last position of each digit; at the first position where a larger digit appears later, swap with its last occurrence: O(d).

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Find last occurrence of max digit for each position — O(n). Walk-through:"

last = {digit: index for digit,index in enumerate digits from right} -> last occurrence index of each digit.

For i in 0..n-1:

d = digits[i].

For larger in 9..d+1:

If last[larger] > i: <- a larger digit appears later

Swap digits[i] and digits[last[larger]].

Return int(''.join(digits)).

Return num (no beneficial swap found).

WHY scan left-to-right: leftmost position benefits most from a swap (higher place value).

WHY largest digit appearing later: swap at position i with the rightmost occurrence of the

largest digit > digits[i] that appears after i.

WHY rightmost: if multiple occurrences of max digit, rightmost gives max after swap.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _670_MaximumSwap:
    def maximumSwap(self, num):
        digits = list(str(num))
        last = {int(d): i for i, d in enumerate(digits)}
        for i, d in enumerate(digits):
            for bigger in range(9, int(d), -1):
                if last.get(bigger, -1) > i:
                    j = last[bigger]
                    digits[i], digits[j] = digits[j], digi
ts[i]
                    return int(''.join(digits))
        return num
```

PHASE 5 TEST & DEBUG

"Let me trace the greedy choice and show why that choice can't hurt the optimal result."

num=2736: last={2:0,7:1,3:2,6:3}. i=0(2): bigger=9..3 not in last. bigger=7: last[7]=1>0->swap digits[0] and digits[1]. -> 7236 ok

num=9973: i=0(9): no bigger. i=1(9): no bigger. i=2(7): bigger=9: last[9]=1, but 1<2->no swap. bigger=8: not in. etc. -> 9973 ok (no better swap)

Last[d]=rightmost index ensures we swap with the rightmost occurrence of each bigger digit

PHASE 5 TEST & DEBUG

Edge: num=0 -> return 0 ok. Single digit: no swap needed. num=1993 -> last biggest on right -> swap i=1(9 already biggest). -> 9913? No: 9 is biggest, no swap -> 1993 ok.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(d*9)	**O(d)**
=O(d)	For d<=10, effectively O(1)

Follow-up: Maximum Swap II — multiple swaps allowed; just sort digits descending.

Follow-up: Minimum Swap — find first digit larger than a later smaller digit.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Greedy | LeetCode #846 | 8 of 12 | 8/12

Hand of Straights

QUESTION

Alice has some number of cards and she wants to rearrange the cards into groups so that each group is of size groupSize, and consists of groupSize consecutive cards.

Given an integer array hand where hand[i] is the value written on the ith card and an integer groupSize, return true if she can rearrange the cards, or false otherwise.

Example 1	
Input	`hand = [1,2,3,6,2,3,4,7,8], groupSize = 3`
Output	`true`
Explanation	*Alice's hand can be rearranged as [1,2,3],[2,3,4],[6,7,8]*

Example 2	
Input	`hand = [1,2,3,4,5], groupSize = 4`
Output	`false`
Explanation	*Alice's hand can not be rearranged into groups of 4.*

CONSTRAINTS

- 1 <= hand.length <= 10^4
- 0 <= hand[i] <= 10^9
- 1 <= groupSize <= hand.length
- Note: This question is the same as 1296: https://leetcode.com/problems/divide-array-in-sets-of-k-consecutive-numbers/

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Can hand be rearranged into groups of groupSize consecutive cards each? Right?"

"A few quick questions:

`len(hand) % groupSize must be 0, else immediately false? Yes."`

"Let me walk: hand=[1,2,3,6,2,3,4,7,8], groupSize=3 -> [1,2,3],[2,3,4],[6,7,8] -> true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
while True:
    take the smallest remaining card and find (by
        scanning) the next groupSize - 1
        consecutive values, removing them
    fail if one is missing
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Scans the hand for each card. Count the cards in a hash map and walk the values in sorted order, starting a group at each smallest remaining value and decrementing the next ones: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Greedy using sorted counter — O(n log n). Walk-through:"

Count frequencies. Sort unique cards.

For each unique card value v in sorted order:

`count = freq[v]. If count == 0: skip.`

For i in 0..k-1:

If freq[v+i] < count: return False (not enough of v+i to complete count groups).

freq[v+i] -= count (consume 'count' copies of each card in the group).

WHY process in sorted order: a group must be consecutive. Processing v first forces

us to form all groups starting at v before moving to v+1.

WHY count groups of size k starting at v: if we have count copies of v, we MUST form

exactly count groups containing v (no other group can use v).

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import Counter
import heapq
class _846_HandOfStraights:
    def isNStraightHand(self, hand, groupSize):
        if len(hand) % groupSize != 0:
            return False
        count = Counter(hand)
        min_heap = list(count.keys())
        heapq.heapify(min_heap)
        while min_heap:
            smallest = min_heap[0]
            if count[smallest] == 0:
                heapq.heappop(min_heap)
                continue
            needed = count[smallest]
            for card in range(smallest, smallest + groupSi
ze):
                if count[card] < needed:
                    return False
```

```
                count[card] -= needed
            if count[smallest] == 0:
                heapq.heappop(min_heap)
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace the greedy choice and show why that choice can't hurt the optimal result."

hand=[1,2,3,6,2,3,4,7,8], groupSize=3: count={1:1,2:2,3:2,4:1,6:1,7:1,8:1}. heap=[1,2,3,4,6,7,8].

smallest=1,needed=1: check 1,2,3 each have count>=1 -> subtract 1 each. count={1:0,2:1,3:1,4:1,...}.

pop 1. smallest=2,needed=1: check 2,3,4 -> subtract. count={2:0,3:0,4:0,...}. pop 2.

smallest=6,needed=1: check 6,7,8 -> subtract -> all 0. -> True ok

Processing the smallest remaining card first ensures we never 'strand' a card with no smaller neighbor to start a group

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	O(n)

Follow-up: Divide Array in Sets of K Consecutive Numbers (#1296) — identical problem, different name.

Follow-up: if order must be preserved (no rearranging), different DP/greedy needed.

MEDIUM | Greedy | LeetCode #954 | 9 of 12 | 9/12

Array of Doubled Pairs

QUESTION

Given an integer array of even length arr, return true if it is possible to reorder arr such that arr[2 * i + 1] = 2 * arr[2 * i] for every 0 <= i < len(arr) / 2, or false otherwise.

Example 1

Input	`arr = [3,1,3,6]`
Output	`false`

Example 2

Input	`arr = [2,1,2,6]`
Output	`false`

Example 3

Input	`arr = [4,-2,2,-4]`
Output	`true`
Explanation	*We can take two groups, [-2,-4] and [2,4] to form [-2,-4,2,4] or [2,4,-2,-4].*

CONSTRAINTS

- 2 <= arr.length <= $3 * 10^4$
- arr.length is even.
- 10^5 <= arr[i] <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Can we reorder arr so every two elements form a pair where arr[2i+1] = 2*arr[2i]?

Return true if such a pairing exists. Right?"

"A few quick questions:

Values can be negative? Yes — e.g. [-2,-4] works since -4=-2*2.

The array has even length? Yes — guaranteed."

"Let me walk: arr=[4,-2,2,-4] -> pair (-2,-4) and (2,4) -> true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
try every way to pair the numbers so that each pair
    is (x, 2x)
return True if one pairing uses all of them
```

TIME	SPACE
exponential	O(n)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

Tries all pairings. Count the values; process them in order of absolute value, and for each x with count c, require at least c copies of 2x: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the combinatorial search.

Greedy: sort by absolute value. Process the smallest absolute value element first.

For each element x, we need to find and consume its pair 2*x.

A Counter tracks remaining elements. If 2*x is not available, return False.

Time: O(n log n). Space: O(n) for the Counter.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _954_ArrayOfDoubledPairs:
    def canReorderDoubled(self, arr):
        from collections import Counter
        count = Counter(arr)
        for x in sorted(count.keys(), key=abs):
            if count[x] > 0:
                if count[2 * x] < count[x]:
                    return False
                count[2 * x] -= count[x]
                count[x] = 0
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace the greedy choice and show why that choice can't hurt the optimal result."

arr=[4,-2,2,-4]: count={4:1,-2:1,2:1,-4:1}.

sorted by abs: [-2,-4,2,4].

x=-2: count[-2]=1>0. count[-4]=1>=1 ok. count[-4]=0, count[-2]=0.

x=-4: count[-4]=0, skip. x=2: count[2]=1>0. count[4]=1>=1 ok. count[4]=0.

x=4: count[4]=0, skip. -> True ok

arr=[3,1,3,6]: count={3:2,1:1,6:1}.

x=1: count[2]=0 < count[1]=1 -> False ok

Sorting by abs value handles negatives correctly — -2 pairs with -4

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(n)**
sort + linear scan	Counter

The abs-value sort is the key insight for handling negatives.

Follow-up: if 'double' is replaced by 'k times' — same greedy, check count[k*x].

Follow-up: count the number of valid pairings — DP or combinatorics on the counts.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Greedy | LeetCode #1647 | 10 of 12 | 10/12

Minimum Deletions to Make Character Frequencies Unique

QUESTION

A string s is called good if there are no two different characters in s that have the same frequency.

Given a string s, return the minimum number of characters you need to delete to make s good.

The frequency of a character in a string is the number of times it appears in the string. For example, in the string "aab", the frequency of 'a' is 2, while the frequency of 'b' is 1.

Example 1

Input	`s = "aab"`
Output	`0`
Explanation	*s is already good.*

Example 2

Input	`s = "aaabbbcc"`
Output	`2`
Explanation	*You can delete two 'b's resulting in the good string "aaabcc". Another way it to delete one 'b' and one 'c' resulting in the good string "aaabbc".*

Example 3

Input	`s = "ceabaacb"`
Output	`2`
Explanation	*You can delete both 'c's resulting in the good string "eabaab". Note that we only care about characters that are still in the string at the end (i.e. frequency of 0 is ignored).*

CONSTRAINTS

- $1 <= s.length <= 10^5$
- s contains only lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the minimum number of character deletions to make all character frequencies unique. Right?"

"A few quick questions:

Two chars can have the same frequency after deletions if... no, each frequency must be unique.

Frequency 0 means the char is fully deleted — is 0 considered a unique frequency? No, 0 doesn't count."

PHASE 1 CLARIFY

"Let me walk: s='aaabbbcc' -> freqs=[3,3,2]. One 3 must change: delete one 'b' -> [3,2,2] -> delete one 'b' -> [3,2,1] ok, deletions=2. Or delete one 'c' -> [3,3,1] still conflict, delete one more -> 2 anyway ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while two letters share a frequency:
    reduce one of them by one deletion, and repeat
```

TIME	SPACE
O(n * 26)	O(26)

WHY IT IS SLOW

Reduces one step at a time. Sort the frequencies in descending order; each must be at most min(previous - 1, own), counting deletions for the difference: O(26 log 26 + n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Greedy: for each frequency that isn't unique, decrement until unique — O(n log n). Walk-through:"

`freq = Counter(s). freq_count = Counter(freq.values()).`

For f from max_freq down to 1:

If freq_count[f] > 1: <- more than one character has this frequency

`extra = freq_count[f] - 1. <- all but one must change`

`deletions += extra. <- delete one occurrence from each extra character to reduce freq by 1`

freq_count[f-1] += extra. <- those characters now have freq f-1

Return deletions.

WHY decrement by 1: minimizes deletions (each char that needs to change reduces by 1).

WHY freq_count[f-1] gets extra: those chars move to freq f-1 in next iteration.

f=0 is fine (character fully deleted).

Time: O(n + max_freq). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import Counter
class _1647_MinDeletionsCharFreqUnique:
    def minDeletions(self, s):
        freqs = sorted(Counter(s).values(), reverse=True)
        deletions = 0
        used_freqs = set()
        for freq in freqs:
            while freq > 0 and freq in used_freqs:
```

```
                freq -= 1
                deletions += 1
            if freq > 0:
                used_freqs.add(freq)
        return deletions
```

PHASE 5 TEST & DEBUG

"Let me trace the greedy choice and show why that choice can't hurt the optimal result."

s='aaabbbcc': freqs sorted=[3,3,2].

freq=3: not in used -> add 3. freq=3: 3 in used -> decrement to 2, deletions=1. 2 in used -> 1, deletions=2. Add 1.

freq=2: 2 in used -> 1, deletions=3. 1 in used -> 0, deletions=4. 0 not added. Hmm, that's 4?

Wait: freqs=[3,3,2]. used={3}. Second freq=3: 3->2(del=1), 2 not in used->add 2. Third freq=2: 2 in used->1(del=2), add 1. Total=2 ok

Used_freqs prevents duplicate allocation; freq=0 is never added

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n + k^2)$ where k <= 26	**O(k)**

In practice k <= 26 so O(n) total.

Follow-up: if we could also duplicate characters (add instead of delete), min operations differs.

Follow-up: Maximum Erasure Value (#1695) — similar greedy on subarrays.

MEDIUM | Greedy | LeetCode #1899 | 11 of 12 | 11/12

Merge Triplets to Form Target Triplet

QUESTION

A triplet is an array of three integers. You are given a 2D integer array triplets, where triplets[i] = [a_i, b_i, c_i] describes the ith triplet. You are also given an integer array target = [x, y, z] that describes the triplet you want to obtain.

To obtain target, you may apply the following operation on triplets any number of times (possibly zero):

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Merge operation: pick two triplets, replace with elementwise max. Can we reach the target triplet

through merges using a subset of the given triplets? Right?"

"A few quick questions:

A triplet is 'usable' only if NONE of its values exceed the corresponding target value (else it would

corrupt that position via max)? Yes — that's the key insight."

"Let me walk: triplets=[[2,5,3],[1,8,4],[1,7,5]], target=[2,7,5] -> use [2,5,3] and [1,7,5] -> max gives [2,7,5] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets(triplets):
    merge them by taking the max of each position
return True if one equals the target
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Tries every subset. A triplet with any value above the target is useless; merge all the others and check whether every target position is matched: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Greedy scan — O(n). Walk-through:"

For each triplet [a,b,c]:

If a <= target[0] and b <= target[1] and c <= target[2]: mark triplet as usable.

Maxima: max_a = max(max_a, a), max_b = max(max_b, b), max_c = max(max_c, c).

PHASE 3 OPTIMIZE

After scan: check if ANY usable triplet combined with the global maxima can form target:

We can take max values from any triplets independently.

If max_a == target[0] and max_b == target[1] and max_c == target[2]: return True.

But only valid if each max came from a triplet that doesn't exceed target in any field.

Actually: result True iff we can merge triplets to reach exactly target in all positions.

Track: can reach target[i] via maxima of usable triplets only.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1899_MergeTripletsToFormTargetTriplet:
    def mergeTriplets(self, triplets, target):
        a = b = c = 0
        for x, y, z in triplets:
            if x <= target[0] and y <= target[1] and z <=
target[2]:
                a = max(a, x)
                b = max(b, y)
                c = max(c, z)
        return [a, b, c] == target
```

PHASE 5 TEST & DEBUG

"Let me trace the greedy choice and show why that choice can't hurt the optimal result."

triplets=[[2,5,3],[1,8,4],[1,7,5]], target=[2,7,5]:

[2,5,3]: 2<=2,5<=7,3<=5 -> valid. a,b,c = 2,5,3.

[1,8,4]: 8>7 -> invalid, skip.

[1,7,5]: 1<=2,7<=7,5<=5 -> valid. a,b,c = max(2,1)=2, max(5,7)=7, max(3,5)=5.

[a,b,c]=[2,7,5] == target -> True ok

Skipping triplets with any value exceeding target avoids corrupting an already-achieved coordinate via max

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

Follow-up: Maximum Performance of a Team (#1383) — different greedy with sorting + heap.

Follow-up: if the operation were elementwise MIN instead of MAX, the filtering condition would flip to >= target.

MEDIUM | Greedy | LeetCode #2131 | 12 of 12 | 12/12

Longest Palindrome by Concatenating Two Letter Words

QUESTION

You are given an array of strings words. Each element of words consists of two lowercase English letters.

Create the longest possible palindrome by selecting some elements from words and concatenating them in any order. Each element can be selected at most once.

Return the length of the longest palindrome that you can create. If it is impossible to create any palindrome, return 0.

A palindrome is a string that reads the same forward and backward.

Example 1

Input	`words = ["lc","cl","gg"]`
Output	**6**
Explanation	*One longest palindrome is "lc" + "gg" + "cl" = "lcggcl", of length 6. Note that "clgglc" is another longest palindrome that can be created.*

Example 2

Input	`words = ["ab","ty","yt","lc","cl","ab"]`
Output	**8**
Explanation	*One longest palindrome is "ty" + "lc" + "cl" + "yt" = "tylcclyt", of length 8. Note that "lcyttycl" is another longest palindrome that can be created.*

Example 3

Input	`words = ["cc","ll","xx"]`
Output	**2**
Explanation	*One longest palindrome is "cc", of length 2. Note that "ll" is another longest palindrome that can be created, and so is "xx".*

CONSTRAINTS

- 1 <= words.length <= 10^5
- words[i].length == 2
- words[i] consists of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Two-letter words; select some subset and concatenate into the longest palindrome.

Each word used at most once. Letters are case-sensitive. Right?"

PHASE 1 CLARIFY

"A few quick questions:

Can the same word appear multiple times in the input? Yes.

'lc' and 'cl' can be placed symmetrically? Yes: 'lc...cl'."

"Let me walk: words=['lc','cl','gg'] -> 'lc'+'gg'+'cl' = 'lcggcl' -> len=6 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets(words):
    try every ordering and check whether the
        concatenation is a palindrome
    best = max(best, candidate)
```

TIME	SPACE
O(2^n * n! * L)	O(n * L)

WHY IT IS SLOW

Exponential. Count each word: pair a word with its reverse (4 letters each), and use at most one word like 'aa' (a palindrome by itself) in the middle: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is enumeration.

Greedy with a Counter:

Non-palindromic words (ab!=ba): pair 'ab' with 'ba' — each matched pair contributes 4 chars.

Palindromic words (gg, aa): each pair-of-two contributes 4 chars; one leftover can be the center (+2).

Walk through pairs of words and their reverses; track how many centers are available.

Time: O(n). Space: O(n) Counter."

Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _2131_LongestPalindromeTwoLetterWords:
    def longestPalindrome(self, words):
        from collections import Counter
        count = Counter(words)
        length = 0
        center_used = False
        for word, freq in count.items():
            rev = word[::-1]
            if word == rev:
                pairs = freq // 2
```

```
                length += pairs * 4
                if freq % 2 == 1 and not center_used:
                    length += 2
                    center_used = True
            elif rev in count and word < rev:
                matched = min(freq, count[rev])
                length += matched * 4
        return length
```

PHASE 5 TEST & DEBUG

"Let me trace the greedy choice and show why that choice can't hurt the optimal result."

words=['lc','cl','gg']:

count={lc:1,cl:1,gg:1}.

'gg'=rev: pairs=0, freq%2=1, center_used=False -> length+=2, center_used=True.

'lc' < 'cl'? No. 'cl' < 'lc'? Yes. min(count[cl],count[lc])=1 -> length+=4.

total=2+4=6 ok

words=['ab','ty','yt','lc','cl','ab']: count={ab:2,ty:1,yt:1,lc:1,cl:1}.

'ty'<'yt': min(1,1)=1 -> +4. 'lc'<'cl': +4. 'ab'=rev? No (ab!=ba). 'ba' not in count.

total=8 ok

Processing word < rev avoids double-counting non-palindromic pairs

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
one pass over Counter	Counter

At most one palindromic center word contributes 2 chars — track with center_used flag.

Follow-up: Longest Palindrome (#409) — same structure with single characters.

Follow-up: if we need the actual palindrome string, construct it from paired words.